

“기지 간소화 (base)” 문제 풀이

작성자: 김동현

풀이 설명의 편의상, 문제에서 정의된 ‘기지’는 ‘정점’, ‘통로’는 ‘간선’으로 서술하였다.

부분문제 1

$i, i+1, \dots, j$ 번 정점만 남기기로 했다고 할 때, 어떤 간선이 ‘유지 비용에 관여하는지’는 다음과 동치임을 우선 알 수 있다.

- 해당 간선을 기준으로 양 쪽 컴포넌트 각각에 $\{i, i+1, \dots, j\}$ 에 속하는 정점이 존재한다.

트리 구조의 특성상 위 조건을 만족하는 간선은 반드시 포함시켜야 하며, 반대로 위 조건을 만족하지 않는 간선은 포함시킬 필요가 없음을 알 수 있다.

부분문제 1을 해결하기 위해서는 각 (i, j) 쌍에 대해 유지 비용에 관여하는 간선을 $O(N)$ 에 구하면 된다. 이는 정점 하나를 루트로 잡고 DFS를 하면서 각 간선 아래쪽 서브트리에 있는 $\{i, i+1, \dots, j\}$ 에 속하는 정점 개수를 세 주면 $O(N)$ 만에 각 간선이 유지 비용에 관여하는지 아닌지를 알 수 있다.

부분문제 2

각 간선에 대해, 해당 간선이 유지 비용에 관여하게 되는 (i, j) 쌍의 개수를 세는 방식으로 접근하자. 쌍의 개수와 간선 길이를 곱한 값을 모든 간선에 대해 더하면 된다.

간선을 고정했을 때, 해당 간선 기준으로 나뉘는 컴포넌트 둘 중 한 쪽을 검은색, 반대쪽을 하얀색으로 색칠했다고 하자. 이제 이 간선이 관여하는 (i, j) 쌍은 $[i, j]$ 구간에 검은색과 하얀색 정점이 모두 존재하는 쌍이다.

간선을 고정한 뒤 색칠하는 것은 DFS로 $O(N)$ 에 할 수 있고, 각 i 에 대해 검은색과 하얀색 정점을 모두 포함하게 되는 최소 j 를 구해나가는 것 역시 $O(N)$ 에 수행할 수 있다.

부분문제 3

0번 노드를 루트로 잡게 되면, 각 간선에 대해 아래쪽 서브트리에 속하는 점들의 번호가 구간의 형태로 나타남을 알 수 있다. 즉, 0번부터 $N-1$ 번 정점까지의 색깔이 하얀색 구간 - 검은색 구간 - 하얀색 구간의 형태로 나타난다. 각각의 길이를 A, B, C 라 하면 검은색과 하얀색을 모두 포함하는 (i, j) 쌍의 개수는 $\frac{N(N+1)}{2} - \frac{A(A+1)}{2} - \frac{B(B+1)}{2} - \frac{C(C+1)}{2}$ 와 같은 수식으로 구할 수 있다.

부분문제 4

다음 연산을 지원하는 자료구조를 생각하자.

- 정점 하나를 검은색으로 칠한다. (초기에는 모든 정점이 하얀색이다)
- 검은색과 하얀색 정점을 모두 포함하는 (i, j) 쌍의 개수를 구한다.

위 연산을 지원하는 자료구조를 빠르게 작동하도록 구현할 수 있다면, 트리가 선형 구조이므로 점을 하나씩 검은색으로 칠하면서 모든 간선에 대해 해당 간선이 관여하는 (i, j) 쌍의 개수를 구할 수 있다.

검은색으로 칠해진 연속한 정점 번호의 구간들을 관리하는 `std::set` 자료구조를 이용하는 것이 가장 간단하게 구현할 수 있는 방법이며, 이 방법을 사용할 시 업데이트(검은색 칠하기)에 $O(\log N)$ 시간, 쌍 개수 구하기에 $O(1)$ 시간이 걸리게 되어 총 시간복잡도 $O(N \log N)$ 에 부분문제를 해결할 수 있다.

부분문제 5

부분문제 4에서 구현한 자료구조를 일반적인 트리에서도 적용할 수 있다. HLD(Heavy-Light Decomposition) 내지는 Small-to-Large Trick으로 불리는 테크닉을 사용하면 된다.

한 정점을 루트로 잡고 DFS를 수행하면서, 각 정점마다 “해당 정점이 루트인 서브트리”에 해당하는 자료구조를 구한다고 하자. 각 정점 v 에서 다음과 같은 작업을 수행한다.

- v 의 자식들 각각에 해당하는 자료구조를 모두 구한다. 이들 중 검은색 점이 가장 많이 칠해진 자식을 c_{max} 라고 하자.
- v 에 해당하는 자료구조는 우선 c_{max} 에 해당하는 자료구조를 그대로 가져온 뒤 c_{max} 를 제외한 다른 자식들에서 칠해진 검은 점을 v 에 해당하는 자료구조에 하나씩 옮겨 칠해 준다.
 - 자료구조를 “그대로 가져온다”라는 것은 복사를 하는 것이 아니라, `std::swap` 내지는 포인터 등을 이용해 reference 자체를 $O(1)$ 만에 가져오는 것을 의미한다.
- 자료구조에 v 를 새로 검은색으로 칠한다.

이런 식으로 DFS를 수행하면 검은색 점을 칠하게 되는 횟수가 $O(N \log N)$ 이 된다. 검은색 점을 처음에 칠하는 횟수는 정점당 한 번이므로 $O(N)$ 이고, 검은색 점이 한 자료구조에서 다른 자료구조로 옮겨갈 때 그 자료구조의 크기가 2배 이상 증가하므로 옮겨다니는 횟수는 $O(\log N)$ 이 되기 때문이다. 즉, 시간복잡도 $O(N \log^2 N)$ 에 문제를 해결할 수 있다.

“회의실 2 (meeting2)” 문제 풀이

작성자: 이종영

부분문제 1

어떤 회의들의 조합에 대해 필요한 회의실의 수는 해당 회의들을 구간으로 한 구간 그래프의 연결 컴포넌트의 수와 같다. 이는 미리 회의들을 정렬해 두었다면 $O(N)$ 에 구할 수 있다.

이제 회의를 취소하는 모든 $N!$ 개의 방법을 시도해보면 $O(N \cdot N!)$ 에 해결할 수 있다.

부분문제 2

각 날에 중요한 것은 남아있는 회의들의 종류와 현재까지 비용의 합이다. 남아있는 회의들의 종류가 같은 경우 비용의 합이 최소인 경우들만 고려하면 된다. 즉, 유의미한 상태의 개수의 수를 $O(2^N)$ 개로 줄일 수 있어, 동적 프로그래밍을 이용해 문제를 $O(N \cdot 2^N)$ 에 해결할 수 있다.

부분문제 3

앞으로는 회의를 제거하는 것이 아닌, 하나씩 추가하는 것으로 생각한다. 또한, 설명의 편의를 위해 회의 대신 각 정점이 구간인 구간 그래프의 측면에서 설명한다.

최종 그래프에서의 연결 컴포넌트에 대해, 해당 컴포넌트의 정점들을 연결성을 유지하며 하나씩 추가하는 것이 가능하다. 또한, 어떤 날의 연결 컴포넌트의 수는 최종 그래프의 연결 컴포넌트들 중, 그 날에 하나 이상의 정점이 포함된 컴포넌트들의 수 이상이다.

이로부터 크기의 내림차순으로 연결 컴포넌트들간의 순서를 정한 후, 현재 컴포넌트의 정점들을 모두 추가한 후 다음 컴포넌트로 넘어가는 방법이 최적의 방법임을 알 수 있다. 크기가 같은 연결 컴포넌트들끼리는 자유롭게 순서를 정할 수 있으므로, 어떤 크기 c 를 가진 연결 컴포넌트의 수가 k 개 있다면, 최종 답에는 $k!$ 이 곱해져야 한다.

이제, 하나의 연결 컴포넌트에 대해 연결성을 유지하면서 정점들을 순서대로 추가하는 방법의 수를 세어야 한다. 이는 동적 프로그래밍으로 해결할 수 있다.

$D[l, r]$ 을 $[l, r]$ 구간에 완전히 포함된 구간들을 모두 처리했을 때 가능한 경우의 수라고 정의하자. $[l, r]$ 과 겹치는 시각이 존재하며 $[l, r]$ 에 완전히 포함되지는 않는 어떤 구간 $[s, e]$ 에 대해, $[L, R] = [l, r] \cup [s, e]$ 로 정의하자. $D[l, r]$ 에서 $D[L, R]$ 로 전이를 하기 위해서는 $[L, R]$ 에는 포함되지만 $[l, r]$ 에는 포함되지 않는 구간들에 대한 처리가 필요하다. 이러한 구간의 개수는 미리 $O(N^2)$ 에 각 구간에 포함되는 구간의 수 $S[l, r]$ 을 2차원 부분합과 유사한 방법으로 계산해 주면 $S[L, R] - S[l, r]$ 로 구할 수 있다.

$[s, e]$ 를 제외한 $S[L, R] - S[l, r] - 1$ 개의 구간들은 $[s, e]$ 를 추가한 후 어떤 순간에 추가해도 연결성 및 연결 컴포넌트의 모양에 영향을 미치지 않는다. 따라서 이후 이 구간들을 원하는 위치에 끼워넣는다고 생각하면 답에 $(N - S[L, R] + 1)(N - S[L, R] + 2) \cdots (N - S[l, r] - 1) = \frac{(N - S[l, r] - 1)!}{(N - S[L, R])!}$ 이 곱해짐을 알 수 있다. 즉, $D[L, R]$ 에 $\frac{(N - S[l, r] - 1)!}{(N - S[L, R])!} D[l, r]$ 이 더해진다. 미리 가능한 모든 $[l, r]$ 에 대해 $l(l+1) \cdots r$ 의 값을 전처리로 구하면 하나의 전이를 $O(1)$ 에 할 수 있다. 가능한 $[s, e]$ 의 개수는 $O(N)$ 개가 있으므로 하나의 $[l, r]$ 에서의 전이는 $O(N)$ 이 소요된다. 상태의 수는 $O(N^2)$ 이므로 문제를 $O(N^3)$ 에 해결할 수 있다.

부분문제 4

$[L, R]$ 에 대해, $[l, r] \cup [s, e] = [L, R]$ 로 전이되는 $[l, r]$ 과 $[s, e]$ 의 조합으로 가능한 경우를 생각해 보자. 우선,

$L = S[i]$ 와 $R = E[j]$ 를 만족하는 어떤 구간 i 와 j 가 존재해야 하며 $L \leq S[j], E[i] \leq R$ 를 만족해야 한다. 그렇지 않다면 해당 $[L, R]$ 은 실제로는 존재할 수 없는 경우이다. 위를 만족하는 i 와 j 가 존재한다면 $i = j$ 인 경우와 그렇지 않은 경우를 나누어 해결한다.

$i = j$ 인 경우는 $[L, R] = [s, e] = [S[i], E[i]]$ 인 경우이다. $L < l < r < R$ 을 만족하는 모든 $[l, r]$ 에 대해, $D[L, R]$ 에 $\frac{(N-S[l, r]-1)!}{(N-S[L, R])!} D[l, r]$ 이 더해진다. $(N-S[l, r]-1)!D[l, r]$ 의 합을 구해준 후, $(N-S[L, R])!$ 로 나누어 준다고 생각하면 2차원 부분합을 $D[L, R]$ 값들을 채워나가며 유지한다면 $O(1)$ 에 구할 수 있다. 실제로 나누는 것은 모듈러 역원을 이용해야 한다. 미리 $O(N)$ 에 필요한 역원들을 전처리로 구하면 $O(1)$ 에 구할 수 있다.

$i \neq j$ 인 경우는 $[s, e] = [S[i], E[i]]$ 이거나 $[s, e] = [S[j], E[j]]$ 이다. $[s, e] = [S[i], E[i]]$ 인 경우 $S[i] < l < E[i], r = R$ 을 만족하는 모든 $[l, r]$ 에 대해, $D[L, R]$ 에 $\frac{(N-S[l, r]-1)!}{(N-S[L, R])!} D[l, r]$ 이 더해진다. 이번에도 $(N-S[l, r]-1)!D[l, r]$ 의 합을 구해준 후, $(N-S[L, R])!$ 로 나누어 준다고 생각하면 각 R 에 대한 부분합을 $D[L, R]$ 값들을 채워나가며 유지한다면 $O(1)$ 에 구할 수 있다. $[s, e] = [S[j], E[j]]$ 인 경우도 비슷하게 각 L 에 대한 부분합을 유지하면 $O(1)$ 에 구할 수 있다.

이제 $[L, R]$ 에 대해 가능한 전이들을 $O(1)$ 에 처리할 수 있어, 문제를 $O(N^2)$ 에 해결할 수 있다.

“학생들 (students)” 문제 풀이

작성자: 윤창기

부분문제 1

어떤 그룹에도 들어 있지 않은 학생은 첫날 신고를 하므로 이 경우 답은 1이 되고, 민원을 접수하는 학생의 목록은 모든 구간의 교집합에 속하는 학생들이 된다. 어떤 학생 v 가 민원을 접수하게 되는 과정은 다음과 같다.

- v 가 속해 있는 멘토링 그룹의 집합을 G_v 라고 표기하자. 학생 v 는 G_v 이외의 멘토링 그룹이 존재하지 않는다고 생각한다.
- G_v 의 모든 그룹에 속해 있지 않은 학생 w 가 존재한다. 이러한 학생은 그룹의 정의상 무조건 존재하고, $|G_v \cap G_w| \leq |G_v| - 1$ 이 된다.
- 재귀적으로, v 의 관점에서 w 가 민원을 접수할 것이라고 예상되는 날짜 $d_w := f(w, G_v \cap G_w)$ 를 계산한다. $f(v, S)$ 는 학생 v 가 S 에 속한 멘토링 그룹만 볼 수 있다고 가정할 때 민원을 접수하게 되는 날짜이다.
- 만약 $\min_w d_w$ 날이 지났음에도 민원을 접수하지 않았다면, G_v 이외의 멘토링 그룹이 존재하지 않는다는 가정에 위배된다. 따라서 진실을 깨달은 학생 v 는 다음날에 민원을 접수하며, $f(v, G_v) = \min_w f(w, G_v \cap G_w) + 1$ 이 된다.

가능한 $f(v, S)$ 의 개수는 많아야 $N \cdot 2^M$ 개이므로, 모든 $f(v, S)$ 를 계산하는데 $\mathcal{O}(N^2 2^M)$ 시간이면 충분하다. 동시에, $f(v, S)$ 를 계산하는 과정으로부터 무조건 신고하는 학생이 존재함을 알 수 있다. 즉, 답이 -1인 경우는 존재하지 않는다.

부분문제 2

부분문제 2는 모든 $L[i] = R[i]$ 인 특수한 경우로, 이 경우 서로 다른 $L[i]$ 의 개수가 답이 된다. 이는 $f(v, S)$ 를 구하는 과정을 생각하면 수학적 귀납법으로 증명할 수 있다.

부분문제 3

부분문제 3을 해결하기 위해서는 답의 값인 $\min_v f(v, G_v)$ 를 보다 간단한 형태로 계산할 필요가 있다. 결론부터 말해, v 가 속해있지 않은 멘토링 그룹을 $H_v = G_v^c$ 라고 할 때 문제의 답은 $|H_{v_1} \cup \dots \cup H_{v_k}| = M$ 을 만족하는 k 의 최솟값이고, 가능한 모든 $T := \{v_1, \dots, v_k\}$ 에 대해 학생 $v_i \in T$ 는 k 일차에 민원을 접수한다. H_v 가 구간임을 감안할 때, 조건을 만족하는 T 는 문제에 주어진 모든 구간에 한 번 이상씩은 포함되는, “interval stabbing set” 중 크기가 가장 작은 것으로 이해할 수 있다.

가능한 T 중 크기가 최소인 것을 minimum stabbing set이라고 정의하고, 그 크기를 $m(U)$ 라고 표기하자. U 는 모든 멘토링 그룹들의 집합을 의미하며, 이제 $m(U)$ 에 대한 수학적 귀납법으로 앞서 주장한 명제를 증명할 수 있다.

- $m(U) = 1$ 인 경우는 모든 멘토링 그룹에서 배제된 학생이 존재하므로 1일만에 민원이 접수된다. 학생 v 가 모든 멘토링 그룹에서 배제되는 것과 $H_v = U$ 가 동치임에 주목하자.

- $m(U) > 1$ 인 경우, $m(G_v)$ 는 $m(U) - 1$ 또는 $m(U)$ 로, 각 경우는 v 를 포함하는 minimum stabbing set이 존재하는 경우와 그렇지 않은 경우에 대응된다. 귀납 가정에 의해, $m(G_v) = m(U) - 1$ 이므로 $m(U) - 1$ 일까지 민원을 접수하는 학생이 나오지 않는다면 학생 v 는 진실을 깨닫고 $m(G_v) + 1 = m(U)$ 일에 민원을 접수한다. 그렇지 않은 경우, $m(G_v) = m(U)$ 일이 지나기 전까지는 $G_v = U$ 인 경우와 그렇지 않은 경우를 구분할 수 없으므로 그 학생은 민원을 접수하지 않는다.

따라서 학생 v 마다 $m(G_v)$ 를 계산해주면 문제의 답과 학생 v 의 신고 여부를 알 수 있다. $m(G_v)$ 는 동적 계획법 등을 이용해 $\mathcal{O}(N + M)$ 시간에 구할 수 있으므로 총 $\mathcal{O}(N(N + M))$ 시간에 문제를 해결할 수 있다.

부분문제 4

부분문제 4는 부분문제 2의 확장된 형태로, 겹치지 않게 잡을 수 있는 구간 수의 최댓값이 답이 된다. 이 때 어떤 구간 $[L_1, R_1]$ 이 $[L_2, R_2]$ 를 완전히 포함하면 $[L_1, R_1]$ 을 고려할 필요가 없음을 관찰하면 좋다.

부분문제 5

이 경우 학생 i 를 포함하지 않는 멘토링 그룹들의 번호가 $[0, l]$ 또는 $[r, M - 1]$ 의 꼴로 나타나게 된다. 따라서 $[0, l]$ 번 구간의 minimum stabbing set의 크기를 모든 l 에 대해 동적 계획법으로 구해 두고, 마찬가지로 $[r, M - 1]$ 번 구간의 minimum stabbing set 크기를 구해 두면 두 minimum stabbing set의 크기를 더한 것이 답이 된다. 학생 i 로 인해 두 구간이 나뉘어 있기 때문에 답이 겹치지 않음에 주목하자. 시간복잡도는 동적 계획법에 필요한 $\mathcal{O}(N + M)$ 이다.

부분문제 6

부분문제 4에서 관찰할 수 있듯 다른 구간을 포함하는 구간을 모두 제거할 수 있다. 이 때 남은 구간들을 끝점에 따라 정렬하면 부분문제 5와 같은 형태가 되므로 동일하게 문제를 해결할 수 있다. 시간복잡도는 정렬 및 불필요한 구간 제거까지 포함하여 $\mathcal{O}(N + M)$ 이고, $\mathcal{O}((N + M) \log(N + M))$ 풀이도 충분히 만점을 받을 수 있도록 제한이 설정되었다.

“팀 만들기 (teambuilding)” 문제 풀이

작성자: 김세빈

부분문제 1

남학생과 여학생을 고르는 모든 방법을 확인하고 최댓값을 구하면 된다. 시간 복잡도는 $O(NMQ)$ 이다.

부분문제 2

앞으로의 설명의 편의를 위해 여학생들의 번호를 반전하자. 0번 여학생은 $M - 1$ 번 여학생이 되고, $M - 1$ 번 여학생은 0번 여학생이 된다.

시나리오의 수 Q 가 작으므로 하나의 시나리오를 효율적으로 해결하는 것에 집중하자. $L_1[k] = 0$, $R_1[k] = N - 1$, $L_2[k] = 0$, $R_2[k] = M - 1$ 인 경우를 생각하도록 한다.

i 번 남학생과 j 번 여학생이 팀을 이뤘을 때, 팀의 실력을 $C[i][j]$ 라 정의하자. 학생들의 발상 능력이 증가하고 구현 능력이 감소한다는 사실을 이용하면 아래와 같은 성질을 관찰하고 증명할 수 있다.

$0 \leq i_1 < i_2 \leq N - 1$ 이고 $0 \leq j_1 < j_2 \leq M - 1$ 일 때, 다음이 성립한다.

$$C[i_1][j_1] + C[i_2][j_2] \geq C[i_1][j_2] + C[i_2][j_1]$$

증명은 다음과 같이 할 수 있다.

$$\begin{aligned} & (C[i_1][j_1] + C[i_2][j_2]) - (C[i_1][j_2] + C[i_2][j_1]) \\ &= (A_1[i_1] + A_2[j_1])(B_1[i_1] + B_2[j_1]) + (A_1[i_2] + A_2[j_2])(B_1[i_2] + B_2[j_2]) \\ &\quad - (A_1[i_1] + A_2[j_2])(B_1[i_1] + B_2[j_2]) - (A_1[i_2] + A_2[j_1])(B_1[i_2] + B_2[j_1]) \\ &= (A_1[i_1]B_2[j_1] + A_2[j_1]B_1[i_1] + A_1[i_2]B_2[j_2] + A_2[j_2]B_1[i_2]) \\ &\quad - (A_1[i_1]B_2[j_2] + A_2[j_2]B_1[i_1] + A_1[i_2]B_2[j_1] + A_2[j_1]B_1[i_2]) \\ &= (A_1[i_2] - A_1[i_1])(B_2[j_2] - B_2[j_1]) + (A_2[j_1] - A_2[j_2])(B_1[i_1] - B_1[i_2]) \\ &\geq 0 \end{aligned}$$

위 성질을 만족하는 배열을 Monge array라고 한다. Monge array는 최적화 문제에서 흔하게 나타나며, 다양하게 활용할 수 있는 성질들을 가지고 있어 중요하다. 일례로, Monge array인 $C[i][j]$ 는 다음과 같은 성질을 가지고 있다:

i 가 주어졌을 때, $C[i][j]$ 의 값이 최대가 되도록 하는 j 중 가장 작은 것을 j_i^* 라 하자. 만약 $0 \leq i_1 < i_2 \leq N - 1$ 이라면, $j_{i_1}^* \leq j_{i_2}^*$ 가 성립한다. 다시 말해, 팀의 실력을 최대화하는 여학생의 번호는 남학생의 번호가 증가함에 따라 단조증가한다.

증명은 다음과 같이 할 수 있다. 귀류법을 사용해 $j_{i_1}^* > j_{i_2}^*$ 라 가정하자. 정의에 의해, $C[i_1][j_{i_1}^*] > C[i_1][j_{i_2}^*]$ 이고, $C[i_2][j_{i_2}^*] \geq C[i_2][j_{i_1}^*]$ 이다 (등호에 유의하자). 이때 $C[i_1][j_{i_2}^*] + C[i_2][j_{i_1}^*] < C[i_1][j_{i_1}^*] + C[i_2][j_{i_2}^*]$ 가 되어 Monge array의 조건에 모순이다.

모든 $0 \leq i \leq N - 1$ 에 대해 j_i^* 를 구하여 부분문제 2를 해결할 수 있다. j_i^* 가 단조증가하므로, 이는 분할 정복 최적화(Divide and Conquer Optimization)라는 최적화 기법을 통해 가능하다.

시간 복잡도는 $O(Q(N + M) \log N)$ 이다.

부분문제 3

부분문제 2에서와 같이 모든 $0 \leq i \leq N - 1$ 에 대해 j_i^* 를 구하면, i 가 주어졌을 때 $C[i][j]$ 의 최댓값을 구할 수 있다. 이 값을 $D[i]$ 라 하자. 부분문제 3에서 각 시나리오의 답은 D 에서의 구간 최대 쿼리 (Range Maximum Query)를 통해 구할 수 있다. 시간 복잡도는 $O((N + M + Q) \log N)$ 이다.

부분문제 4

먼저, 모든 시나리오에서 0번 남학생을 고를 수 있는 경우를 생각해 보자. 즉, 모든 $0 \leq k \leq Q - 1$ 에 대해 $L_1[k] = 0$ 인 경우를 생각해 보자. 또한, $R_1[k] = n$ 인 시나리오만 고려하자. 우리의 목표는 n 을 0부터 $N - 1$ 까지 증가시키며 모든 시나리오의 답을 구하는 것이다.

이번에는 j 가 주어졌을 때, $C[i][j]$ 의 값이 최대가 되도록 하는 $0 \leq i \leq n$ 중 가장 작은 것을 i_j^* 라 하자. 이때, 부분문제 2에서 관찰한 성질에 의해 i_j^* 는 j 에 대해 단조증가한다. i_j^* 의 값이 동일한 j 들을 하나의 구간으로 묶어 생각하면, i_j^* 의 정보를 순서쌍들의 배열 $(a_1, b_1), \dots, (a_t, b_t)$ 로 나타낼 수 있다. 이것의 의미는 구간 $(b_{l-1}, b_l]$ 에 속하는 j 들에 대해 i_j^* 의 값이 a_l 이라는 것이다 ($1 \leq l \leq t$). 이때 $b_0 = 0$, $b_t = M - 1$ 로 정의된다. $a_1 < \dots < a_t$, $b_1 < \dots < b_t$ 가 성립한다는 사실을 알 수 있다.

어떤 j 가 주어졌을 때 i_j^* 의 값을 구하기 위해서는 이 순서쌍들의 배열에서 이분 탐색을 하면 된다. 따라서 하나의 시나리오에 대한 답을 $O(\log M)$ 에 구할 수 있다.

이제 n 이 증가함에 따라 이 순서쌍들의 배열이 어떻게 변화하는지 관찰하면, 배열의 오른쪽 끝부분에서만 변화가 일어난다는 사실을 알 수 있다. 따라서 순서쌍들을 스택으로 관리하여 효율적으로 처리할 수 있다. i_j^* 의 값이 동일한 구간을 계산하는 데는 이분 탐색이 필요하다. 따라서 n 을 0부터 $N - 1$ 까지 증가시키는 데 필요한 시간 복잡도는 $O(N \log M)$ 이다.

이제 $L_1[k] = 0$ 인 경우, 즉 시나리오가 남학생들의 prefix에 대해 주어지는 경우 문제를 $O((N + Q) \log M)$ 에 해결할 수 있다. Prefix에 대해 문제를 해결할 수 있고 이를 뒤집으면 suffix에 대해서도 마찬가지이므로, 분할 정복 (Divide and Conquer) 기법을 활용해 부분문제 4를 $O(N \log N \log M + Q \log M)$ 에 해결할 수 있다. 이는 잘 알려진 방법이며, 간단히 설명하면 다음과 같다.

- $0 \leq m \leq N - 1$ 인 m 을 고르고, m 을 포함하는 모든 시나리오에 대한 답을 구하자. 즉, $L_1[k] \leq m \leq R_1[k]$ 를 만족하는 시나리오에 대한 답을 구하자.
- 구간 $[L_1[k], R_1[k]]$ 에 대한 답을 얻기 위해서는 두 구간 $[L_1[k], m]$ 과 $[m, R_1[k]]$ 에 대해 문제를 해결한 뒤 최댓값을 취하면 된다. 이는 각각 m 에서 시작하는 prefix와 suffix이므로, 위에서 설명한 방법과 같이 해결할 수 있다.
- 이제 남은 시나리오들은 모두 m 을 포함하지 않는 시나리오들이다. 즉, 구간 $[0, m - 1]$ 또는 $[m + 1, N - 1]$ 에 완전히 포함되는 시나리오들이다. 따라서 두 구간 $[0, m - 1]$ 과 $[m + 1, N - 1]$ 에 대해 같은 과정을 반복하면 된다.
- 각 과정에 대해, m 을 구간의 중점으로 잡으면 전체 시간 복잡도에 $O(\log N)$ 이 곱해진다.

분할 정복 대신 세그먼트 트리를 활용하여도 유사한 방법으로 해결할 수 있다. 이 경우 시간 복잡도는 $O(N \log M + Q \log N \log M)$ 이고, fractional cascading이라는 최적화 기법을 통해 $O(N \log M + Q(\log N + \log M))$ 로 줄일 수 있다.

부분문제 5

부분문제 4에서는 모든 시나리오에 대해 가능한 여학생의 번호가 유일하므로, 즉 $L_2[k] = R_2[k]$ 이므로, 순서쌍들의 배열 $(a_1, b_1), \dots, (a_t, b_t)$ 에서 단 한 번의 이분 탐색을 통해 시나리오의 답을 구할 수 있었다. 전체 문제에서는 가능한 여학생의 번호 또한 구간으로 주어지기 때문에, 이를 해결하기 위해서는 몇 가지 과정이 더 필요하다.

순서쌍들의 배열 $(a_1, b_1), \dots, (a_t, b_t)$ 에서 이분 탐색을 통해 $L_2[k] \leq b_l$ 이 성립하는 최소의 $1 \leq l \leq t$ 과 $b_r < R_2[k]$ 가 성립하는 최대의 $1 \leq r \leq t$ 을 구할 수 있다. 이때, 시나리오를 통해 주어진 구간 $[L_2[k], R_2[k]]$ 를 다음과 같이 여러 개의 구간으로 분할할 수 있다. 각 구간에 대한 답 중 최댓값이 시나리오의 답이 된다.

$$[L_2[k], b_l], (b_l, b_{l+1}], \dots, (b_{r-1}, b_r], (b_r, R_2[k]]$$

각 구간에 대한 답은 다음과 같이 구할 수 있다.

- 구간 $[L_2[k], b_l]$ 에서는 팀의 실력을 최대화하는 남학생의 번호가 a_l 로 고정된다. 이는 부분문제 4에 주어진 상황에서 남녀를 바꾼 것과 같다. 따라서 부분문제 4와 동일한 방법으로 해결할 수 있다.
- 구간 $(b_r, R_2[k]]$ 에서는 팀의 실력을 최대화하는 남학생의 번호가 a_{r+1} 로 고정된다. 마찬가지로 부분문제 4와 동일한 방법으로 해결할 수 있다.
- $l < s \leq r$ 에 대해, 구간 $(b_{s-1}, b_s]$ 에서 팀의 실력을 최대화하는 남학생의 번호는 a_s 이다. a_s 번 남학생과 구간 $(b_{s-1}, b_s]$ 에서 고른 여학생이 구성할 수 있는 팀의 실력의 최댓값을 $D[s]$ 로 정의하자. $D[s]$ 의 값은 위에서와 마찬가지로 부분문제 4와 동일한 방법으로 해결할 수 있다. 한편 $D[s]$ 는 시나리오, 즉 $L_2[k]$ 와 $R_2[k]$ 의 영향을 받지 않는 값이다. 따라서 순서쌍들의 배열 $(a_1, b_1), \dots, (a_t, b_t)$ 을 구성하는 과정에서 D 값을 미리 전처리할 수 있다. 스택에서 push 또는 pop이 일어날 때마다 해당하는 위치의 값을 다시 계산해 주면 된다. 이제 시나리오에 대한 답을 구할 때는 D 에서 구간 $[l+1, r]$ 에 대한 구간 최대 쿼리(Range Maximum Query)를 통해 구간 $(b_l, b_{l+1}], \dots, (b_{r-1}, b_r]$ 을 한번에 처리할 수 있다.

위에서는 다루지 않았지만 $l = r$ 이거나 $l = r + 1$ 인 경우도 존재할 수 있어 따로 처리해야 한다. 자세한 설명은 생략한다.

부분문제 4에서는 하나의 시나리오에 대한 답을 $O(\log M)$ 에 구할 수 있었다. 부분문제 4의 풀이를 사용하는 횟수를 세어 보면, 전체 문제를 해결하는 시간복잡도가 $O((N + M) \log N \log M + Q \log N)$ 라는 것을 알 수 있다.

분할 정복 대신 세그먼트 트리를 활용하여도 유사한 방식으로 해결할 수 있다. 이 경우 시간 복잡도는 $O((N + M) \log N \log M + Q \log N \log M \log(N + M))$ 또는 $O((N + M) \log N \log M + Q \log^2(N + M))$ 이다.